AI Content Generator

Project Report

Automatically generate Emails, Advertisements, Captions & Product Descriptions using AI

Submitted by

Pooja Gupta

1. Introduction

In today's digital world, content creation is a daily necessity for businesses, students, and professionals alike. Writing emails, creating social media captions, drafting advertisements, and producing product descriptions all require time, creativity, and consistent quality. Many people, especially those without strong writing skills, find this challenging.

This project presents an AI-powered Content Generator — a web-based tool that takes user input (topic, content type, tone, and word limit) and automatically produces high-quality, relevant written content. The system is built using Flask for the backend and integrates with the Groq API (Llama 3.3 model) for natural language generation.

2. Problem Statement

Manual content creation faces several real-world challenges:

- Time-consuming: Writing good content from scratch takes significant effort.
- Requires creativity and writing skills: Not everyone can write compelling copy.
- Inconsistent quality: Rushed content often lacks polish and professionalism.
- Volume demands: Businesses need large quantities of content regularly.

These challenges lead to delays in communication, poor-quality marketing, and reduced audience engagement. This project directly addresses these pain points by automating content creation with AI.

3. Objectives

- Automatically generate content based on user-provided inputs.
- Support multiple content types: Email, Advertisement, Caption, and Product Description.
- Allow tone selection to match the context (Formal, Casual, Professional, Friendly, Persuasive).
- Maintain quality and relevance in generated output.
- Save time and effort for users with limited writing skills.
- Provide a clean, easy-to-use web interface.

4. Technology Stack

Component	Technology Used

Backend Framework	Python — Flask
AI / NLP Model	Groq API — Llama 3.3 70B Versatile
Frontend	HTML5, CSS3, Jinja2 Templates
Fonts	Quicksand, Pacifico (Google Fonts)
Environment	python-dotenv (.env file for API key)
IDE	Visual Studio Code
Language	Python 3.14

5. System Design & Architecture

The system follows a simple client-server architecture:

- The user fills in the web form (topic, content type, tone, word limit) and submits it.
- Flask receives the POST request and extracts the form values.
- A structured prompt is built using the inputs and sent to the Groq API.
- The Llama 3.3 model processes the prompt and returns relevant, high-quality content.
- Flask passes the output back to the HTML template and displays it to the user.

The API key is stored securely in a .env file and loaded at runtime using python-dotenv, keeping it out of the source code entirely.

6. Features

- Four content types: Email, Advertisement, Caption, Product Description.
- Five tone options: Formal, Casual, Professional, Friendly, Persuasive.
- Custom word limit control (30–500 words).
- Copy to clipboard button on generated output.
- Error handling with user-friendly messages.
- Secure API key management via .env file.
- Responsive, clean UI with a consistent pink design theme.

7. Screenshots & Output Samples

The following screenshots demonstrate the working application across different content types and tones.

7.1 Application Interface (Home Screen)

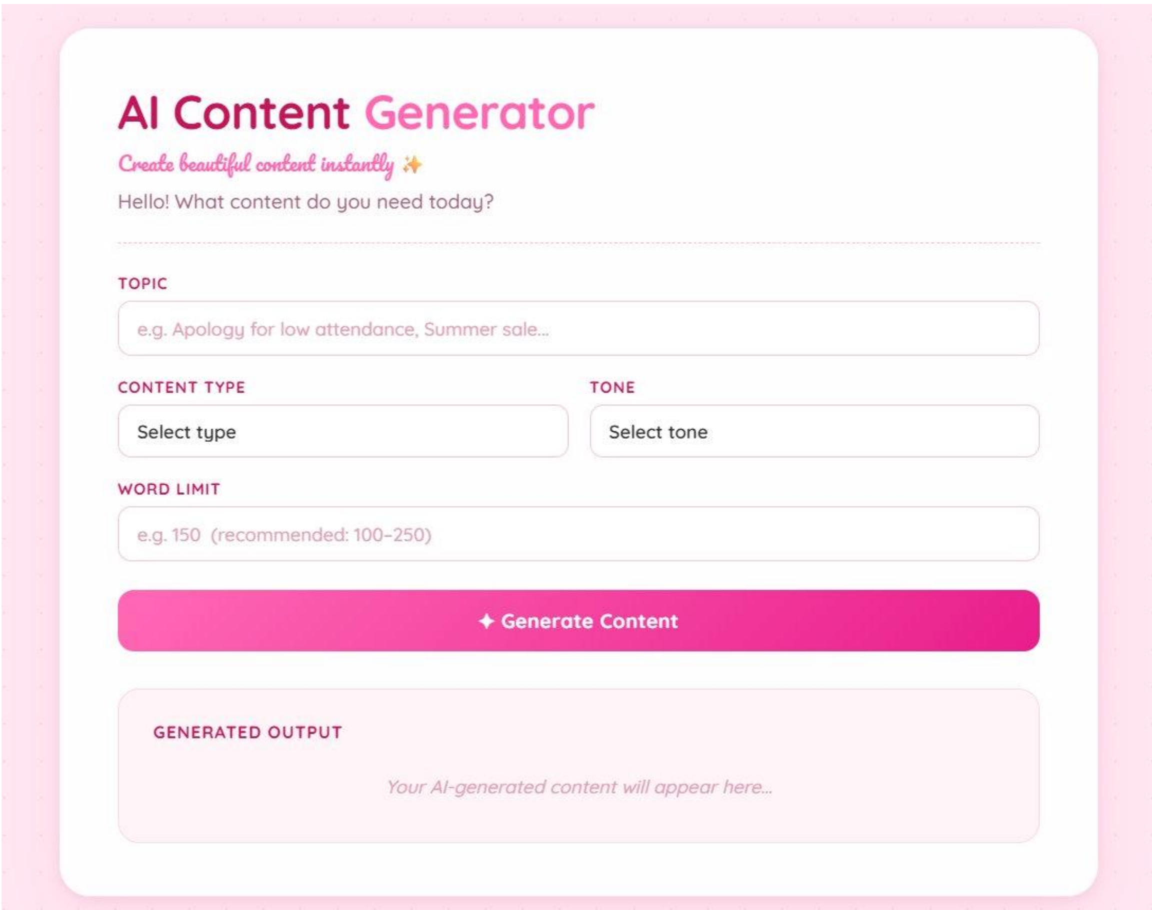

Figure 1: Main UI — empty state showing the form fields

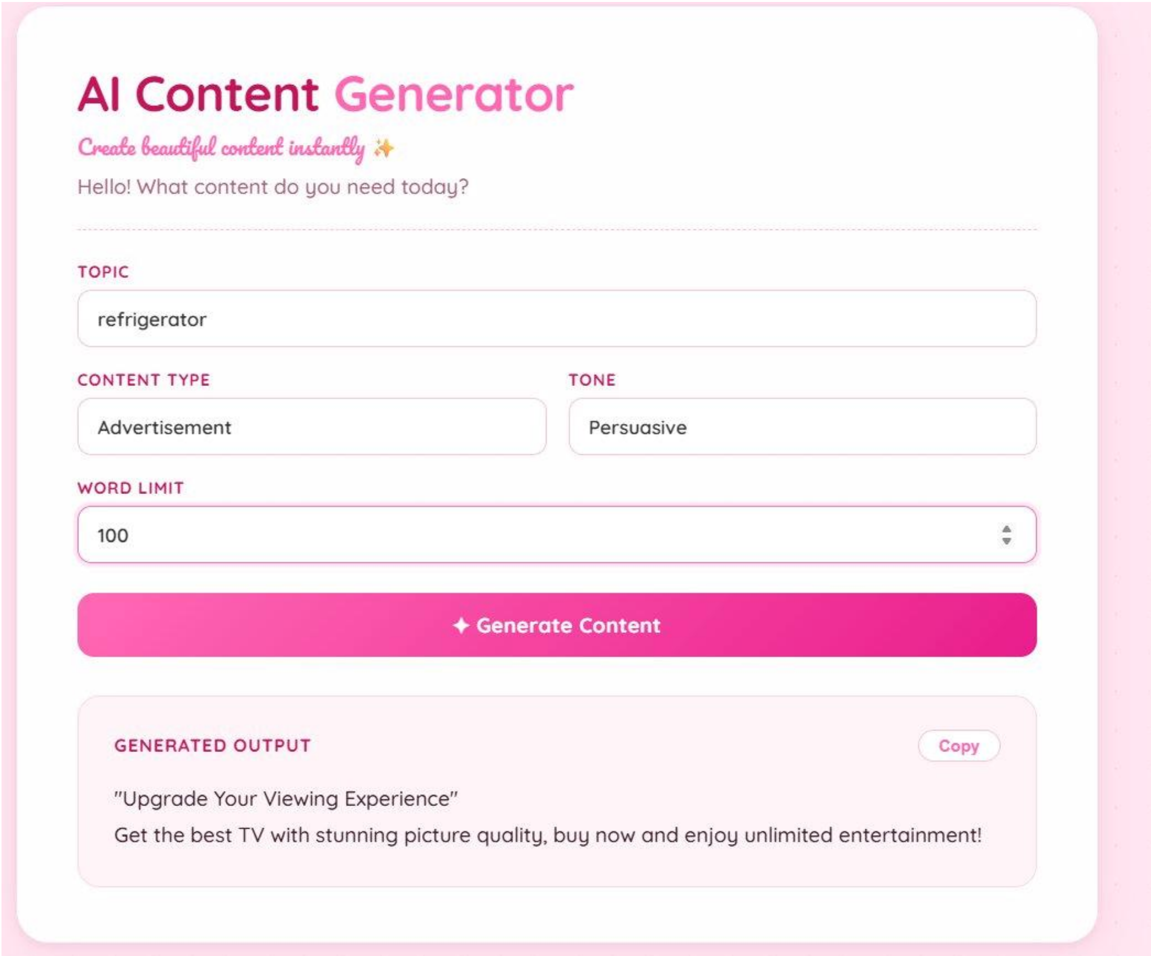

Figure 2: UI with selections filled in (Caption, Friendly tone)

7.2 Email Generation

Topic: "Apology for low attendance" | Tone: Formal | Words: 100

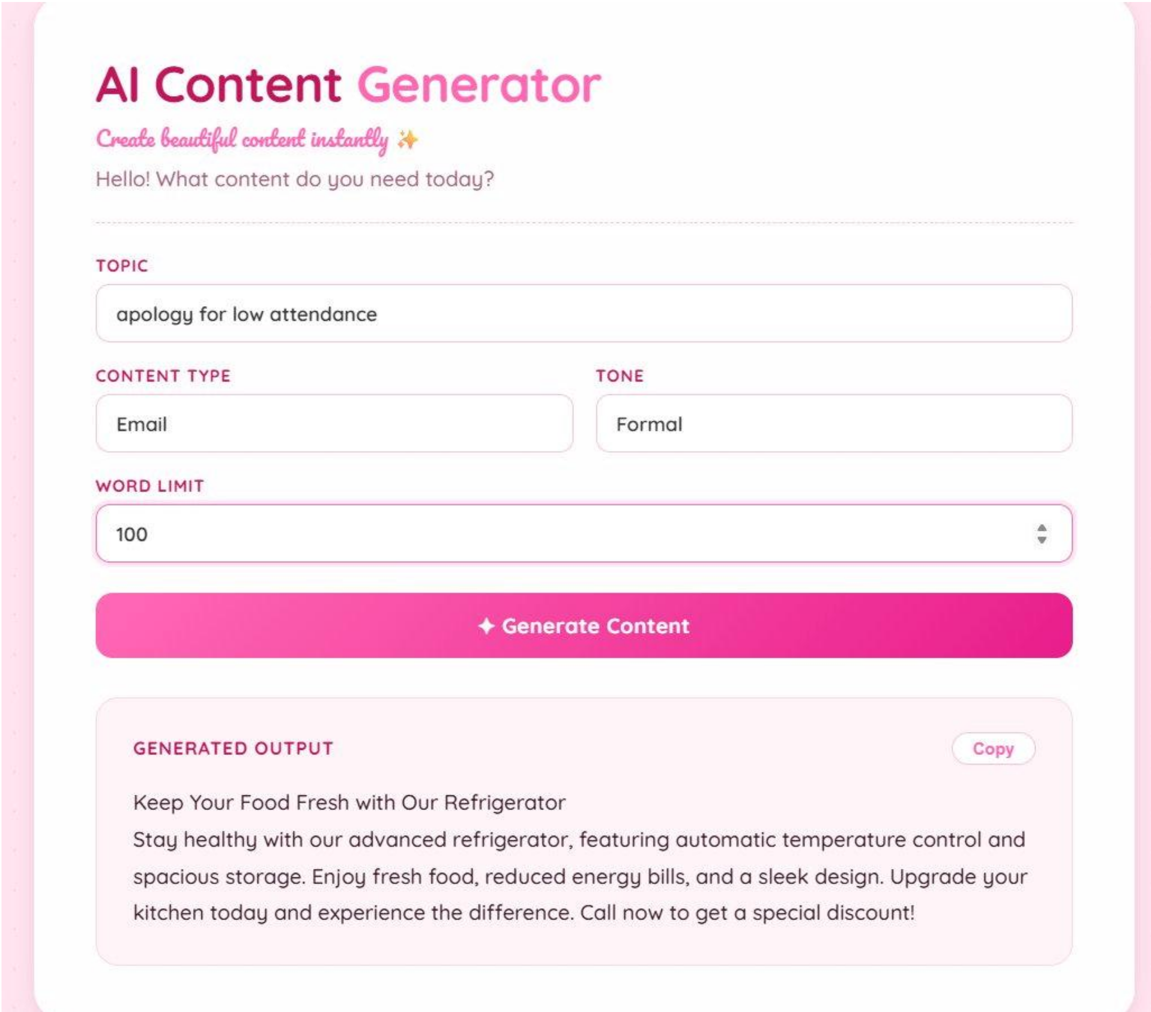

Figure 3: Formal email generated for apology for low attendance

7.3 Advertisement Generation

Topic: "Refrigerator" | Tone: Persuasive | Words: 100

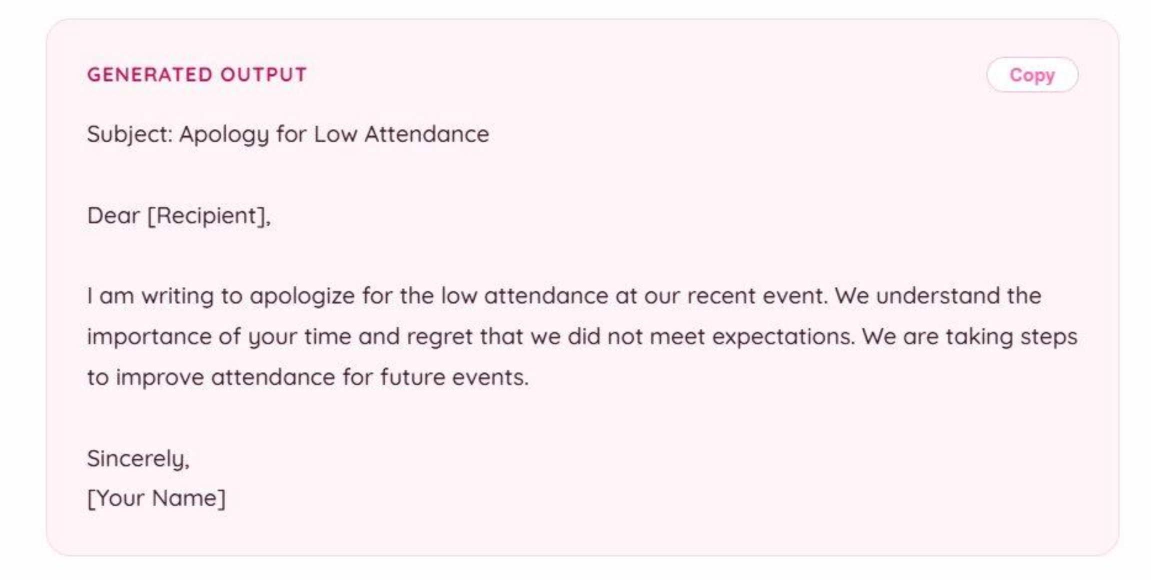

Figure 4: Persuasive advertisement for a refrigerator

7.4 Product Description Generation

Topic: "Mobile phone" | Tone: Professional | Words: 100

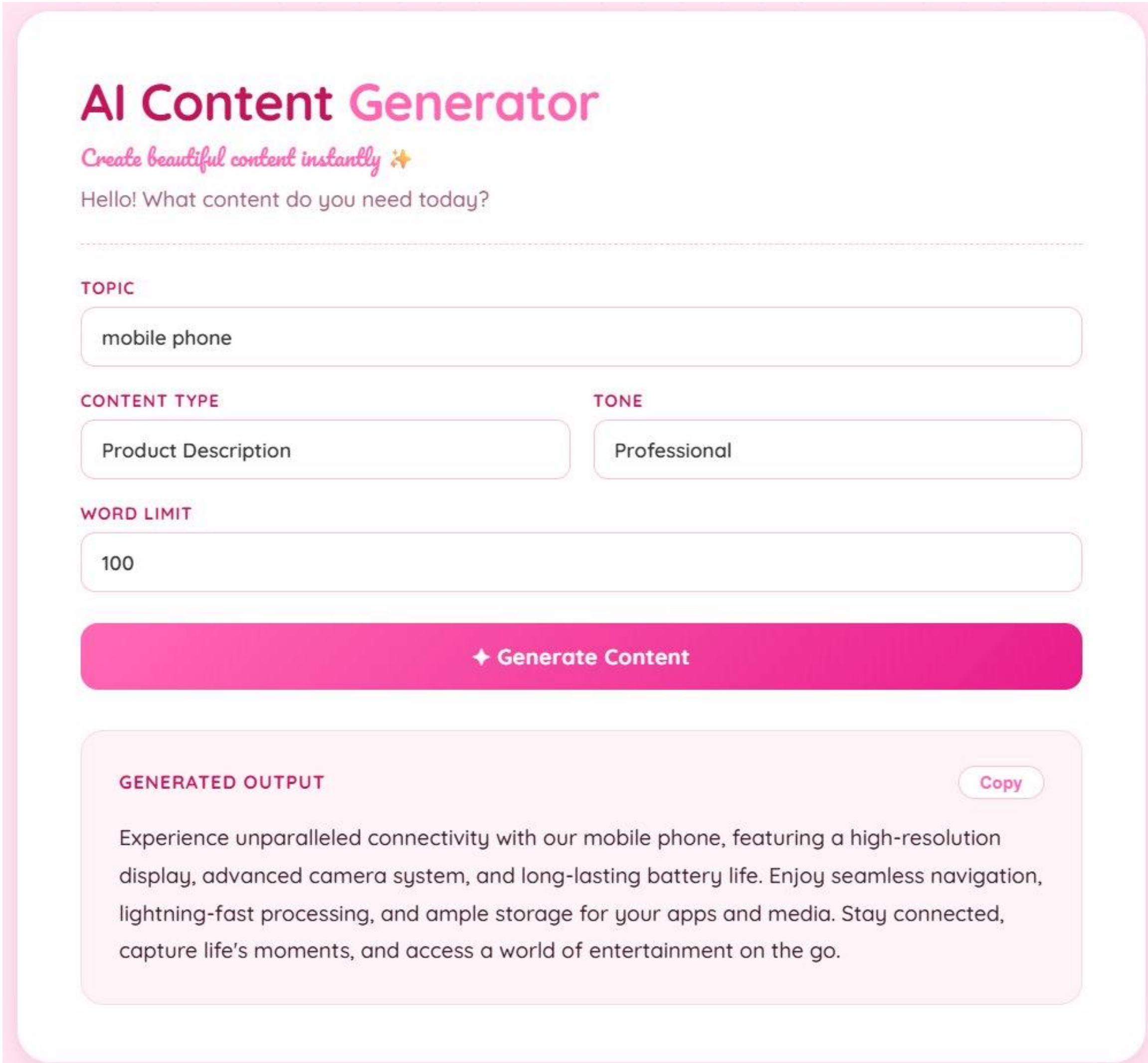

Figure 5: Professional product description for a mobile phone

7.5 Caption Generation

Topic: "Couple photo both hug each other" | Tone: Friendly | Words: 50

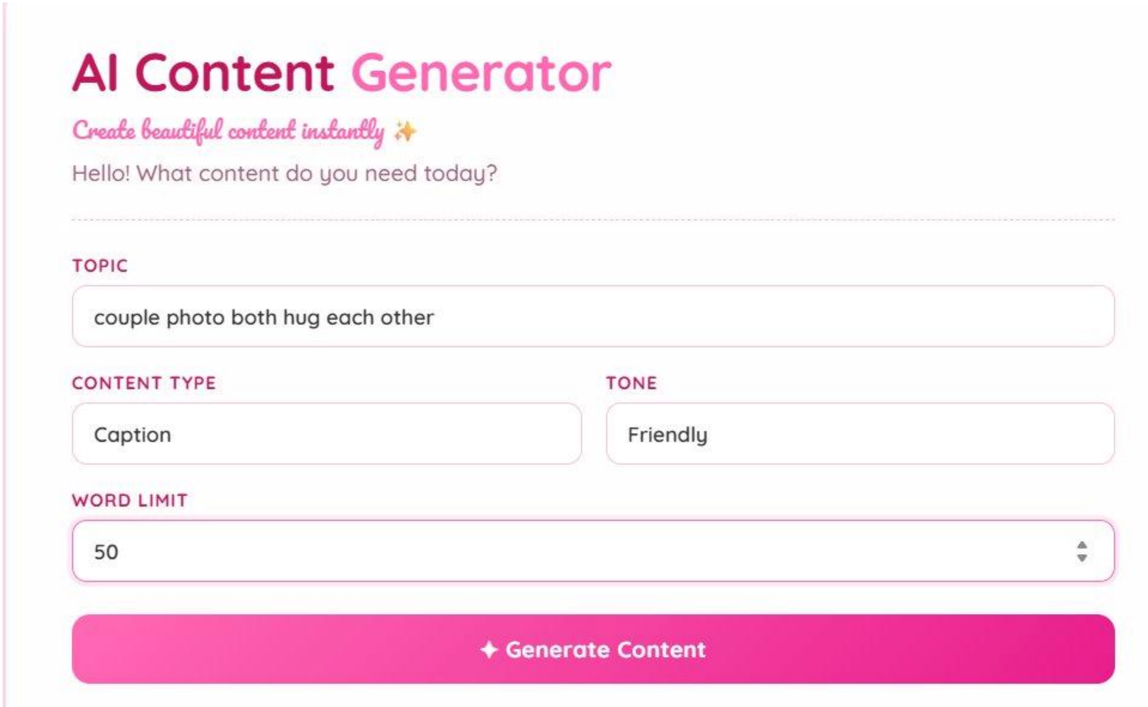

Figure 6: Caption generated with hashtags for a couple photo

7.6 Output Detail Views

Figure 7: Caption output with hashtags — close-up view

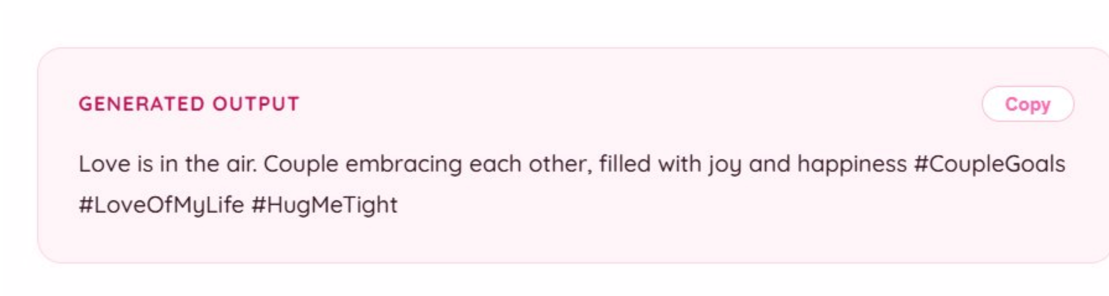

Figure 8: Refrigerator advertisement output — close-up view

8. How It Works

Step-by-step flow of the application:

1. User opens the web interface at http://127.0.0.1:5000.
2. User enters a topic, selects content type, tone, and word limit.
3. On clicking "Generate Content", the form data is sent to Flask via POST request.
4. Flask builds a detailed prompt incorporating all inputs and sends it to the Groq API.
5. The Llama 3.3 70B model returns relevant, human-quality content.
6. The output is displayed on the same page with a Copy button for convenience.

9. Challenges Faced

- Initial use of distilgpt2 model produced random, incoherent outputs. Solved by switching to the Groq API with the Llama 3.3 instruction-following model.
- The llama3-8b-8192 model was decommissioned mid-development. Updated to llama-3.3-70b-versatile which resolved the issue.
- API key management: Learned to use .env files with python-dotenv to keep credentials secure and out of source code.
- Flask app variable not defined error: Corrected the order of declarations (app = Flask(__name__) must precede all route definitions).

10. Conclusion

This project successfully demonstrates how AI can be integrated into a practical web application to solve a real-world problem. The AI Content Generator is capable of producing relevant, high-quality content across multiple categories with different tonal styles — tasks that would otherwise take significant time and skill.

The project helped develop hands-on skills in Python web development (Flask), API integration, prompt engineering, frontend design, and secure coding practices. It reflects a complete development cycle: from identifying a problem to designing, building, debugging, and delivering a working solution.

— End of Report —